

## Code for real data study

```
#####  
# REAL DATA APPLICATION  
# Polynomial Approximation Approach to Testing  
# Covariate--Error Independence in GPLR  
#  
# Data: millerkeith2018data1.csv  
#  
# Proposed statistic:  
#  
#  $T_{\{n,r,d\}} =$   
#  $n \times \int [$   
#  $F_n(u,w) - F_{\{n,U\}}(u) F_{\{n,W\}}(w)$   
#  $]^2 dF_n(u,w)$   
#  
# where  
#  $U_i = P_r(Z_i)$   
#  $W_i = \Delta^d Y_i$   
#  
# Bootstrap calibration follows the paper.  
#####  
  
rm(list = ls())  
  
#####  
# 1. PACKAGES  
#####  
  
packages <- c(  
  "readr",  
  "dplyr"  
)  
  
for (p in packages) {  
  if (!requireNamespace(p, quietly = TRUE)) {  
    install.packages(p)  
  }  
}  
  
library(readr)  
library(dplyr)
```

```
#####
# 2. READ THE WIND-PLANT DATA
#####

data_file <- "millerkeith2018data1.csv"

dat <- read_csv(
  data_file,
  show_col_types = FALSE
)

cat("\n===== \n")
cat("DATA INFORMATION\n")
cat("===== \n")

cat("Observations :", nrow(dat), "\n")
cat("Variables      :", ncol(dat), "\n\n")

print(names(dat))

#####
# 3. CHECK VARIABLES
#####

required_vars <- c(
  "Longitude",
  "Latitude",
  "InstCapMWi",
  "AreaKM2",
  "NetGENe_2016_MWe"
)

missing_vars <- setdiff(
  required_vars,
  names(dat)
)

if (length(missing_vars) > 0) {
  stop(
    paste(
      "The following variables are missing:",
      paste(missing_vars, collapse = ", ")
    )
  )
}
```

```

    )
  )
}

#####
# 4. SELECT VARIABLES
#####

dat <- dat %>%
  select(all_of(required_vars))

#####
# 5. REMOVE MISSING / NON-FINITE VALUES
#####

dat <- dat %>%
  filter(
    is.finite(Longitude),
    is.finite(Latitude),
    is.finite(InstCapMWi),
    is.finite(AreaKM2),
    is.finite(NetGENe_2016_MWe)
  )

cat("\nFinal sample size =", nrow(dat), "\n")

#####
# 6. DEFINE RESPONSE AND COVARIATES
#####

# Response

Y <- dat$NetGENe_2016_MWe

# Covariates

Z <- dat %>%
  select(
    InstCapMWi,
    Longitude,
    Latitude,
    AreaKM2
  )

```

```

)

#####
# 7. STANDARDIZE COVARIATES
#####

# Standardization is used only for numerical stability
# in constructing the polynomial basis.

Z.std <- as.data.frame(
  scale(Z)
)

#####
# 8. COMPLETE POLYNOMIAL BASIS
#####

# This function constructs a complete polynomial basis
# of degree r for p covariates.
#
# For r = 1:
#   1, z1, z2, ..., zp
#
# For r = 2:
#   1, zi, zi*zj, zi^2
#
# For r = 3:
#   all terms of total degree <= 3
#
# The basis dimension is:
#   choose(p+r, r)

complete_polynomial_basis <- function(Z, r) {

  Z <- as.data.frame(Z)

  p <- ncol(Z)

  if (r < 0 || r != floor(r)) {
    stop("r must be a non-negative integer.")
  }

```

```

result <- data.frame(
  intercept = rep(1, nrow(Z))
)

if (r == 0) {
  return(result)
}

# Generate exponent combinations
# whose total degree is <= r.

exponent_list <- list()

generate_exponents <- function(
  current,
  position,
  remaining
) {

  if (position == p) {

    for (degree in 0:remaining) {

      exponent_list[[length(exponent_list) + 1]] <-
        c(current, degree)
    }

    return(NULL)
  }

  for (degree in 0:remaining) {

    generate_exponents(
      c(current, degree),
      position + 1,
      remaining - degree
    )
  }

  return(NULL)
}

```

```

exponent_list <- list()

generate_exponents(
  current = numeric(0),
  position = 1,
  remaining = r
)

# Remove the all-zero exponent because the intercept
# has already been included.

exponent_list <- exponent_list[
  sapply(
    exponent_list,
    function(e) sum(e) > 0
  )
]

for (j in seq_along(exponent_list)) {

  exponent <- exponent_list[[j]]

  term <- rep(1, nrow(Z))

  term_name <- character(0)

  for (k in seq_len(p)) {

    if (exponent[k] > 0) {

      term <- term *
        Z[[k]]^exponent[k]

      if (exponent[k] == 1) {

        term_name <- c(
          term_name,
          names(Z)[k]
        )

      } else {

```

```

        term_name <- c(
            term_name,
            paste0(
                names(Z)[k],
                "^",
                exponent[k]
            )
        )
    }
}

result[[paste(
    term_name,
    collapse = "*"
)]] <- term
}

return(result)
}

#####
# 9. CHECK POLYNOMIAL BASIS DIMENSIONS
#####

cat("\n===== \n")
cat("POLYNOMIAL BASIS DIMENSIONS \n")
cat("===== \n")

for (r in c(1, 2, 3)) {

    Btmp <- complete_polynomial_basis(
        Z.std,
        r
    )

    cat(
        "r =", r,
        ": basis dimension =",
        ncol(Btmp),
        "\n"
    )
}

```

```

}

#####
# 10. FIT POLYNOMIAL APPROXIMATION
#####

fit_polynomial_model <- function(
  Y,
  Z,
  r
) {

  basis <- complete_polynomial_basis(
    Z,
    r
  )

  fit <- lm(
    Y ~ . - 1,
    data = cbind(
      Y = Y,
      basis
    )
  )

  U <- as.numeric(
    fitted(fit)
  )

  residuals <- as.numeric(
    residuals(fit)
  )

  return(
    list(
      model = fit,
      basis = basis,
      U = U,
      residuals = residuals
    )
  )
}

```

```
#####
# 11. FORWARD DIFFERENCE
#####

# Delta^d Y_i =
#
# sum_{k=0}^d (-1)^(d-k)
# choose(d,k) Y_{i+k}

forward_difference <- function(
  Y,
  d
) {

  n <- length(Y)

  if (d < 1 || d != floor(d)) {
    stop("d must be a positive integer.")
  }

  if (n <= d) {
    stop("d is too large for the sample size.")
  }

  W <- numeric(n - d)

  for (i in seq_len(n - d)) {

    value <- 0

    for (k in 0:d) {

      value <- value +
        (-1)^(d - k) *
        choose(d, k) *
        Y[i + k]
    }

    W[i] <- value
  }
}
```

```

    return(W)
}

#####
# 12. ALIGN U AND W
#####

# Since W_i uses Y_i,...,Y_{i+d}, the corresponding
# polynomial approximation is taken for the starting
# observations:
#
# U_1,...,U_{n-d}

align_U_W <- function(
  U,
  W,
  d
) {

  nW <- length(W)

  U.aligned <- U[seq_len(nW)]

  return(
    list(
      U = U.aligned,
      W = W
    )
  )
}

#####
# 13. EMPIRICAL CDF
#####

empirical_cdf_values <- function(
  x,
  points
) {

  #  $F_n(x) = n^{-1} \sum I(X_i \leq x)$ 

```

```

supply(
  points,
  function(z) {
    mean(x <= z)
  }
)
}

#####
# 14. CRAMER--VON MISES INDEPENDENCE STATISTIC
#####

# The statistic is
#
#  $T =$ 
#  $n \times \int$ 
#  $[$ 
#  $F_n(u, w)$ 
#  $- F_{n,U}(u) F_{n,W}(w)$ 
#  $]^2$ 
#  $dF_n(u, w)$ 
#
# Since  $dF_n$  places mass  $1/n$  at each observed pair,
# this becomes
#
#  $T =$ 
#  $\sum_i$ 
#  $[$ 
#  $F_n(U_i, W_i)$ 
#  $- F_{n,U}(U_i) F_{n,W}(W_i)$ 
#  $]^2$ 

cvm_independence_statistic <- function(
  U,
  W
) {

  if (length(U) != length(W)) {
    stop("U and W must have the same length.")
  }

  n <- length(U)

```

```

if (n < 2) {
  stop("At least two observations are required.")
}

#####
# Marginal empirical CDFs
#####

F_U <- rank(
  U,
  ties.method = "max"
) / n

F_W <- rank(
  W,
  ties.method = "max"
) / n

#####
# Joint empirical CDF
#####

F_joint <- numeric(n)

for (i in seq_len(n)) {

  F_joint[i] <- mean(
    U <= U[i] &
    W <= W[i]
  )
}

#####
# Cramer-von Mises statistic
#####

Tstat <- n * mean(
  (
    F_joint -
    F_U * F_W
  )^2
)

```

```

    )

    return(Tstat)
}

#####
# 15. FAST VERSION OF THE STATISTIC
#####

# The direct version above is easy to verify.
# This version uses ranks and is considerably faster.

cvm_independence_statistic_fast <- function(
    U,
    W
) {

    n <- length(U)

    if (length(W) != n) {
        stop("U and W must have the same length.")
    }

    #####
    # Marginal empirical CDFs
    #####

    RU <- rank(
        U,
        ties.method = "max"
    )

    RW <- rank(
        W,
        ties.method = "max"
    )

    F_U <- RU / n
    F_W <- RW / n

    #####
    # Joint empirical CDF

```

```
#####

# For each observation i:
#
# F(U_i,W_i)
#
# = proportion of observations with
# U_j <= U_i and W_j <= W_i

ord <- order(U)

W_sorted <- W[ord]

F_joint <- numeric(n)

for (ii in seq_len(n)) {

  i <- ord[ii]

  F_joint[i] <-
    mean(W_sorted <= W[i])
}

#####
# Statistic
#####

Tstat <- n * mean(
  (
    F_joint -
      F_U * F_W
  )^2
)

return(Tstat)
}

#####
# 16. VERIFY THE TWO IMPLEMENTATIONS
#####

set.seed(123)
```

```

test_U <- rnorm(100)

test_W <- rnorm(100)

T1 <- cvm_independence_statistic(
  test_U,
  test_W
)

T2 <- cvm_independence_statistic_fast(
  test_U,
  test_W
)

cat("\n=====\\n")
cat("STATISTIC IMPLEMENTATION CHECK\\n")
cat("=====\\n")

cat("Direct statistic :", T1, "\\n")
cat("Fast statistic   :", T2, "\\n")
cat(
  "Difference       :",
  abs(T1 - T2),
  "\\n"
)

#####
# 17. BOOTSTRAP CALIBRATION
#####

# Under H0, U and W are independently resampled from
# their empirical marginal distributions.

bootstrap_cvm_test <- function(
  U,
  W,
  B = 2000,
  alpha = 0.05,
  seed = 20260906
) {

```

```

set.seed(seed)

n <- length(U)

if (length(W) != n) {
  stop("U and W must have the same length.")
}

#####
# Observed statistic
#####

observed <- cvm_independence_statistic_fast(
  U,
  W
)

#####
# Bootstrap statistics
#####

bootstrap_statistics <- numeric(B)

for (b in seq_len(B)) {

  # Independent resampling from the two empirical
  # marginal distributions.

  U.star <- sample(
    U,
    size = n,
    replace = TRUE
  )

  W.star <- sample(
    W,
    size = n,
    replace = TRUE
  )

  bootstrap_statistics[b] <-
    cvm_independence_statistic_fast(

```

```

        U.star,
        W.star
    )

    if (b %% 250 == 0) {

        cat(
            "Bootstrap iteration:",
            b,
            "/",
            B,
            "\n"
        )
    }
}

#####
# Bootstrap critical value
#####

critical_value <- quantile(
    bootstrap_statistics,
    probs = 1 - alpha,
    names = FALSE
)

#####
# Bootstrap p-value
#####

p_value <- (
    1 +
    sum(
        bootstrap_statistics >= observed
    )
) / (B + 1)

#####
# Monte Carlo standard error
#####

MCSE <- sqrt(

```

```

    p_value *
      (1 - p_value) /
      (B + 1)
  )

#####
# Decision
#####

decision <- ifelse(
  p_value < alpha,
  "Reject H0",
  "Do not reject H0"
)

return(
  list(
    statistic = observed,
    critical_value = critical_value,
    p_value = p_value,
    MCSE = MCSE,
    decision = decision,
    bootstrap_statistics =
      bootstrap_statistics
  )
)
}

#####
# 18. MAIN REAL-DATA ANALYSIS
#####

# We examine several polynomial degrees and
# finite-difference orders.

r_values <- c(
  1,
  2,
  3
)

d_values <- c(

```

```

1,
2,
3
)

B <- 2000

alpha <- 0.05

#####
# 19. STORAGE
#####

results <- list()

counter <- 1

#####
# 20. LOOP OVER r AND d
#####

for (r in r_values) {

  cat("\n\n#####\n")
  cat("Polynomial degree r =", r, "\n")
  cat("#####\n")

  #####
  # Polynomial approximation
  #####

  polynomial_fit <- fit_polynomial_model(
    Y = Y,
    Z = Z.std,
    r = r
  )

  U.full <- polynomial_fit$U

  #####
  # Model fit information
  #####

```

```

model_summary <- summary(
  polynomial_fit$model
)

cat(
  "R-squared =",
  model_summary$r.squared,
  "\n"
)

cat(
  "Adjusted R-squared =",
  model_summary$adj.r.squared,
  "\n"
)

#####
# Difference orders
#####

for (d in d_values) {

  cat("\n-----\n")
  cat(
    "Running r =",
    r,
    ", d =",
    d,
    "\n"
  )
  cat("-----\n")

  #####
  # Forward difference
  #####

  W <- forward_difference(
    Y,
    d
  )
}

```

```
#####
# Align U and W
#####

aligned <- align_U_W(
  U = U.full,
  W = W,
  d = d
)

U <- aligned$U
W <- aligned$W

#####
# Bootstrap test
#####

test <- bootstrap_cvm_test(
  U = U,
  W = W,
  B = B,
  alpha = alpha,
  seed = 20260906 +
    100 * r +
    d
)

#####
# Save result
#####

results[[counter]] <- data.frame(
  n = length(U),
  r = r,
  d = d,
  Basis_Dimension =
    ncol(
      polynomial_fit$basis
    ),
  R_squared =
    model_summary$r.squared,
  Adjusted_R_squared =
```

```

        model_summary$adj.r.squared,
Test_Statistic =
        test$statistic,
Bootstrap_Critical_Value =
        test$critical_value,
Bootstrap_P_Value =
        test$p_value,
MCSE =
        test$MCSE,
Decision =
        test$decision
)

counter <- counter + 1

#####
# Print result
#####

cat(
  "\nT statistic =",
  test$statistic,
  "\n"
)

cat(
  "Critical value =",
  test$critical_value,
  "\n"
)

cat(
  "Bootstrap p-value =",
  test$p_value,
  "\n"
)

cat(
  "MCSE =",
  test$MCSE,
  "\n"
)

```

```

        cat(
            "Decision =",
            test$decision,
            "\n"
        )
    }
}

#####
# 21. COMBINE RESULTS
#####

results <- bind_rows(results)

#####
# 22. PRINT ALL RESULTS
#####

cat("\n\n=====\\n")
cat("FINAL REAL-DATA RESULTS\\n")
cat("=====\\n")

print(results)

#####
# 23. SAVE COMPLETE RESULTS
#####

write.csv(
    results,
    "RealData_CVM_Test_Results.csv",
    row.names = FALSE
)

#####
# 24. PRIMARY ANALYSIS:  $r = 2$ ,  $d = 2$ 
#####

primary_result <- results %>%
    filter(
        r == 2,

```

```

    d == 2
  )

cat("\n\n=====\\n")
cat("PRIMARY ANALYSIS: r = 2, d = 2\\n")
cat("=====\\n")

print(primary_result)

write.csv(
  primary_result,
  "RealData_Primary_r2_d2.csv",
  row.names = FALSE
)

#####
# 25. SENSITIVITY TO DIFFERENCE ORDER
#####

sensitivity_d <- results %>%
  filter(r == 2) %>%
  select(
    n,
    r,
    d,
    Test_Statistic,
    Bootstrap_Critical_Value,
    Bootstrap_P_Value,
    MCSE,
    Decision
  )

cat("\n\n=====\\n")
cat("SENSITIVITY TO d FOR r = 2\\n")
cat("=====\\n")

print(sensitivity_d)

write.csv(
  sensitivity_d,
  "RealData_Sensitivity_d.csv",
  row.names = FALSE
)

```

```

)

#####
# 26. SENSITIVITY TO POLYNOMIAL DEGREE
#####

sensitivity_r <- results %>%
  filter(d == 2) %>%
  select(
    n,
    r,
    d,
    Basis_Dimension,
    R_squared,
    Adjusted_R_squared,
    Test_Statistic,
    Bootstrap_Critical_Value,
    Bootstrap_P_Value,
    MCSE,
    Decision
  )

cat("\n\n=====\\n")
cat("SENSITIVITY TO r FOR d = 2\\n")
cat("=====\\n")

print(sensitivity_r)

write.csv(
  sensitivity_r,
  "RealData_Sensitivity_r.csv",
  row.names = FALSE
)

#####
# 27. LATEX-FRIENDLY RESULTS
#####

latex_results <- results %>%
  mutate(
    R_squared =
      sprintf("%.4f", R_squared),

```

```

Adjusted_R_squared =
    sprintf("%.4f", Adjusted_R_squared),

Test_Statistic =
    sprintf("%.4f", Test_Statistic),

Bootstrap_Critical_Value =
    sprintf(
        "%.4f",
        Bootstrap_Critical_Value
    ),

Bootstrap_P_Value =
    ifelse(
        Bootstrap_P_Value < 0.001,
        "<0.001",
        sprintf(
            "%.4f",
            Bootstrap_P_Value
        )
    ),

MCSE =
    sprintf("%.4f", MCSE)
)

cat("\n\n=====\\n")
cat("LATEX TABLE DATA\\n")
cat("=====\\n")

print(latex_results)

write.csv(
    latex_results,
    "RealData_LaTeX_Table_Results.csv",
    row.names = FALSE
)

#####
# 28. MODEL FIT TABLE
#####

```

```

model_fit_table <- results %>%
  select(
    r,
    Basis_Dimension,
    R_squared,
    Adjusted_R_squared
  ) %>%
  distinct() %>%
  arrange(r)

cat("\n\n=====\\n")
cat("MODEL FIT TABLE\\n")
cat("=====\\n")

print(model_fit_table)

write.csv(
  model_fit_table,
  "RealData_Model_Fit_Table.csv",
  row.names = FALSE
)

#####
# 29. FINAL MESSAGE
#####

cat("\n\n=====\\n")
cat("REAL-DATA ANALYSIS COMPLETED\\n")
cat("=====\\n")

cat(
  "\\nResults saved in:\\n",
  "RealData_CVM_Test_Results.csv\\n",
  "RealData_Primary_r2_d2.csv\\n",
  "RealData_Sensitivity_d.csv\\n",
  "RealData_Sensitivity_r.csv\\n",
  "RealData_LaTeX_Table_Results.csv\\n",
  "RealData_Model_Fit_Table.csv\\n"
)

##### END #####

```